Claude Code Workflow Tricks

Plugins · Skills · Hooks · MCPs · Patterns for Power Users

mostly Opus 4.6

What This Deck Covers

Environment

- CLI vs. IDE terminal
- Terminal & shortcuts

Workflow Patterns

- Plan-review-code-review
- Prompt templates
- Project prompts & self-improvement

Plugins

- Caveman (token savings)
- Superpowers (structured dev)
- MemPalace (persistent memory)
- Code review & simplify
- Sequential thinking

Integrations & Efficiency

- Context7, Playwright, GitHub
- Hooks for safety
- Cost-conscious usage
- More tricks
- Personal setup

Skills & Subagents

- Highlights & patterns

Target audience: **you already use AI agents**. This is about leveling up.

CLI vs. IDE Terminal

CLI wins for heavy work

- Auto model selection for reasoning tasks
- Full terminal visibility (scrollback, width)
- `Shift+Tab` permission cycling
- Background tasks (`Ctrl+B`)
- Voice mode (`/voice`)
- Piping: `cat file | claude -p "..."`
- Remote control (`/rc`)

IDE wins for focused edits

- Inline diffs with side-by-side review
- `Alt+K` auto-inserts `@file#lines`
- Multiple conversation tabs
- Rewind with visual checkpoints
- See changes in context of surrounding code

Recommendation

CLI for orchestration & long tasks.
IDE for file-focused edits & review.

Terminal & Shortcut Setup

Claude Code shortcuts

- `Shift+Tab` — cycle permission modes
- `Esc` / `Esc+Esc` — stop / rewind
- `Ctrl+O` — toggle verbose (see thinking)
- `Ctrl+B` — background a task
- `Ctrl+G` — open prompt in editor
- `Ctrl+R` — search command history

Useful commands

- `/rename` — rename current session
- `/color` — set accent color
- `/vim` — enable vim keybindings
- `/model` — switch model mid-session

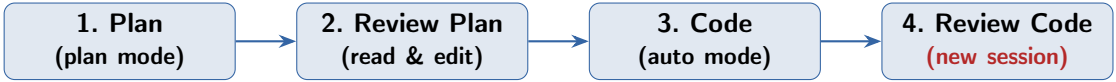
VS Code terminal tips

- Use a Nerd Font for ccstatusline icons
- Increase scrollback: `terminal.integrated.scrollback: 10000`
- Watch for `Ctrl+B` conflict (toggle sidebar)
- Remap or disable VS Code shortcuts that clash

Bash essentials

- `!!` — repeat last command
- `Ctrl+R` — reverse history search
- `Ctrl+L` — clear screen

The Plan-Review-Code-Review Workflow



Why separate sessions?

- Fresh context = fresh eyes
- No accumulated bias from coding phase
- Reviewer catches what the author missed

In practice

- `claude -n "impl-auth"` for coding
- `claude -n "review-auth"` for review
- Or: `/code-review` plugin (4 parallel agents)

Rule

Never skip the plan phase for non-trivial tasks. A 2-minute plan saves 20 minutes of rework.

Prompt Templates & The Interview Trick

Bug fix template

- "Symptom: [X]. Expected: [Y]. Check `@file`. Write a failing test first, then fix."

Feature template

- "Implement [X] following the pattern in `@example`. Plan first, then code."

Refactor template

- "Refactor [X] to [goal]. Keep all tests passing. Show me the plan before changing code."

The "interview me" trick

Before complex tasks, say:

- "Interview me about the requirements before you start."
- Claude asks about edge cases, constraints, preferences
- Write the spec to `SPEC.md`
- Start a *fresh session* focused purely on implementation

Key insight

Output quality is determined by input quality. Templates and interviews enforce good input.

Project Prompts & Self-Improvement

Set up the project

- Clear `CLAUDE.md` (under 200 lines)
- Architecture, constraints, conventions
- Build/test commands Claude can't guess
- `.claude/rules/*.md` for path-specific rules

Track & improve

- "Keep track of your progress as you go"
- "Save what you learned to memory"
- Name sessions: `claude -n "feature-X"`

The meta-improvement loop

Periodically ask Claude:

- "What workflow improvements would you suggest?"
- "Review my CLAUDE.md — what's missing?"
- "What should I automate with hooks or skills?"

Autoresearch pattern

- Modify → evaluate → keep/discard → repeat
- Karpathy's autoresearch: 83 runs overnight, 15 kept, 11% gain
- Any task with a measurable metric

Caveman Plugin

Cut ∼75% output tokens by stripping filler while keeping full technical accuracy.

Install:

```
claude plugin marketplace add JuliusBrussee/caveman
claude plugin install caveman@caveman
```

Three compression levels

- `/caveman lite` — drops filler, keeps grammar
- `/caveman full` — fragments, drops articles
- `/caveman ultra` — maximum telegraphic compression

Why it matters

- Research: brevity constraints improved accuracy by 26 percentage points
- Terse commits, one-line code reviews
- Input compression too (∼46% savings)
- `github.com/JuliusBrussee/caveman`

Superpowers Plugin

Enforced 5-phase development workflow with TDD and subagent-driven code.

Install:

```
/plugin install superpowers@claude-plugins-official
```

The 5 phases

1. **Clarify** requirements
2. **Design** architecture
3. **Plan** implementation steps
4. **Code** with TDD (red-green-refactor)
5. **Verify** with two-stage review

Key features

- Subagent-driven development
- Git worktree isolation
- Systematic debugging with root-cause analysis
- 153k+ stars on GitHub
- By Jesse Vincent (`obra`)

Why use it

Prevents the "just start coding" failure mode. Forces you to think before you type.

MemPalace

Local-first AI memory with 96.6% recall — zero API calls, 100% on your machine.

Install:

```
pip install mempalace
```

How it works

- **Verbatim storage** — no lossy summarization
- Method of loci: Wings → Rooms → Halls → Closets → Drawers
- 29 MCP tools for palace read/write, knowledge graph, agent diaries
- Auto-save hooks before context compression

Why not built-in memory?

- Built-in memory is flat markdown files
- MemPalace: temporal knowledge graph with validity windows
- Cross-session, cross-project recall
- 96.6% on LongMemEval benchmark
- `github.com/MemPalace/mempalace`

Code Review, Simplify & Sequential Thinking

Code Review plugin (Anthropic, free)

- `/code-review` runs 4 parallel agents
- CLAUDE.md compliance, bug scan, git-blame context
- Findings scored 0–100, threshold filters noise
- Install: `/plugin install code-review`

Code Simplify (built-in)

- `/simplify` — post-implementation cleanup
- Preserves functionality, improves clarity
- Best workflow: code → `/code-review` → `/simplify`

Sequential Thinking MCP

- Structured step-by-step reasoning
- Best for: architecture decisions, scope discovery, multi-step planning
- Reported 60–70% time savings on complex features

Setup:

```
claude mcp add seq-think \
  -- npx -y \
  @modelcontextprotocol/\
  server-sequential-thinking
```

Skills & Subagents Highlights

Skills worth knowing

- `/batch` — process multiple files at once
- `/debug` — systematic debugging
- `/claude-api` — Anthropic SDK help
- Custom: write in `.claude/skills/*.md`

Scientific research

- K-Dense-AI/scientific-agent-skills
- 133 skills: genomics, drug discovery, clinical ML, materials science, finance
- Open standard: `agentskills.io`

Subagent power patterns

- `@"reviewer (agent)"` — explicit delegation
- `claude -agent reviewer` — session-wide
- `isolation: worktree` for safe parallel dev
- `Ctrl+B` to background any task

Implement + review pattern

One subagent implements in a worktree.
Another reviews the diff.
Main session merges the results.

Essential MCPs

Context7 — live library docs

```
claude mcp add context7 \
  -- npx -y \
  @upstash/context7-mcp@latest
```

Say "use context7" → current API docs, no hallucination.

Playwright — browser control

```
claude mcp add playwright \
  npx @playwright/mcp@latest
```

Automated testing, screenshots, form filling.

GitHub — repo integration

- `/install-github-app`
- `@claude` in issues/PRs
- Auto PR reviews
- Custom instructions per repo

Deferred tool loading

- MCP tools lazy-load via `ToolSearch`
- Hundreds of tools without context bloat
- Only loaded when actually needed

Hooks for Safety & Automation

Safety hooks (PreToolUse)

- **No force-push**: block `git push -force`
- **No credentials**: block reads of `.env`, `credentials.json`
- **No production**: block deploys to prod
- Exit code 2 = blocked with message

Automation hooks (PostToolUse)

- Auto-format after edits (Prettier, Black)
- Run tests on changed files
- HTTP hook: POST to Slack on completion

Startup briefing (SessionStart)

```
"SessionStart": [{
  "hooks": [{
    "type": "command",
    "command": "node briefing.js"
  }]
}]
```

Print today's tasks, open PRs, failing tests — every time you start.

Config: `.claude/settings.json` (team) | `settings.local.json` (personal) | `~/.claude/settings.json` (global)

Cost-Conscious Usage

Model selection

- `/model` to switch mid-session
- **Opus**: complex architecture, multi-file refactors
- **Sonnet**: most daily coding, good balance
- **Haiku**: quick questions, simple edits, `/btw`

Effort tuning

- `/effort low` for straightforward tasks
- `/effort high` for complex reasoning
- Default (medium) is usually right
- `/cost` — check token usage

Token management

- Caveman plugin: 75% output reduction
- `/btw` for side questions (reuses prompt cache)
- `/context` — visualize what fills your context
- Subagents for exploration (keeps main context clean)

Golden rule of context

`/clear` between *unrelated* tasks.
`/compact` between *related* tasks.

More Tricks

ccstatusline

- Beautiful status bar: model, git branch, tokens, timer
- Powerline themes, custom widgets
- `npx -y ccstatusline@latest`
- `github.com/sirmalloc/ccstatusline`

Piping & scripting

```
cat error.log | claude -p \
  "explain root cause" > out.txt
```

Headless mode for automation, grading scripts, batch processing.

Git worktrees

- `claude -worktree feature-auth`
- Isolated repo copy per task
- No file conflicts between parallel tasks

Path-specific rules

- `.claude/rules/testing.md`
- Loads only when test files are edited
- Keep CLAUDE.md lean, rules specific

Remote control

- `/rc` — continue from phone/browser
- That's it. One command.

Personal Setup: Your AI Assistant

Communication

- **Gmail** — read, draft, search emails
- **Telegram** — reply, send files, reactions
- **Notion** — search, create pages, manage databases

Productivity

- **Google Calendar** — create events, list schedule, suggest meeting times
- **Google Drive** — read, search, create files

How it works

- All via MCP connections
- Runs 100% locally
- Data stays on your machine
- OAuth authentication (one-time setup)
- Available in CLI and IDE

Example workflow

"Check my calendar for tomorrow, draft an email to Alex about the 3pm meeting, and create a Notion page with the agenda."

Summary

Workflow Upgrades

- Plan → review → code → review (new session)
- Prompt templates + "interview me" trick
- Ask Claude to improve your workflow
- `/code-review` → `/simplify` loop
- Golden rule: `/clear` unrelated, `/compact` related

Tools & Integrations

- Caveman (75% fewer tokens)
- Superpowers (structured dev)
- MemPalace (persistent memory)
- Sequential Thinking MCP
- Context7, Playwright, GitHub
- Hooks for safety & automation
- Cost-conscious model/effort selection

The best prompt is a well-structured workflow.

Resources

Plugins & Tools

- **Caveman** — token compression
- **Superpowers** — structured dev
- **MemPalace** — persistent memory
- **ccstatusline** — custom status bar
- **Autoresearch** — experiment loop

Skills & Standards

- **Scientific Agent Skills** — 133 research skills
- **agentskills.io** — open standard
- **Anthropic Skills** — official repo

Learning

- **Anthropic Academy** — free 21-lesson course
- **Claude Code Docs** — official
- **Awesome Claude Code Toolkit**

MCPs

- **MCP Servers** — official collection
- Context7, Playwright, Sequential Thinking
- Gmail, Calendar, Telegram, Drive, Notion

All links clickable in this PDF.